

TREASURE HUNT

Task Implementation — Prompt & Response Record

Tasks 1–4 • Existing game.py

Task 1 — Floor Traps

USER / PROMPT

USER / PROMPT https://github.com/prajnaholla12/24_treasure_hunt.git

Inspect the existing game.py carefully before making changes. Do not rewrite the project or replace the existing game architecture.

Implement Task 1 from the README: add floor traps to the existing dungeon.

Requirements:

- Add visible floor traps in appropriate locations in the existing dungeon.
- When the player steps on a trap, reset the player to their original starting position.
- Show a clear status message such as "You stepped on a trap! Returned to start."
- The trap collision should work reliably with the existing player movement and grid/tile collision system.
- Preserve all existing functionality including movement, key collection, chest interaction, and restart.
- Keep the implementation simple and modular.
- Do not add external dependencies beyond pygame-ce.
- After making the changes, explain exactly what you changed and how I can test the trap functionality.

ASSISTANT / RESPONSE I inspected the actual repository and game.py before making the changes. The README specifically requires Task 1 to add traps without replacing the existing architecture.

What I changed

I kept the existing:

- generate_world() dungeon generation
- Player.move() and its tile/wall collision
- key collection
- chest interaction
- R restart behavior
- GameEngine structure

I only added the trap functionality:

1. Added a TRAP tile type.
2. Added 4 traps to randomly selected walkable floor tiles.
 - Traps aren't placed inside the starting room.
 - They aren't placed on the key/chest because those cells aren't FLOOR.
3. Made traps visibly different
 - Dark red floor tile
 - Red crossed blades/X
4. Stored the original starting position:
`self.start_pos = (sx, sy)`
5. Added trap collision to the existing tile interaction logic:
`if cell == TRAP:`
`self.player.rect.topleft = self.start_pos`
`self.status = "You stepped on a trap! Returned to start."`

This uses the same center-tile detection already used for the key and chest, so it fits the existing grid movement system rather than introducing a separate collision system. The original game checks the player's center tile after `Player.move()`.

6. Restart still works normally. Pressing R generates a new dungeon and new trap positions, just as the existing restart generated a new dungeon.

Updated file

[Download the updated game.py](sandbox:/mnt/data/game.py)

I also ran a Python syntax check on the modified file successfully.

How to test Task 1

The README says the game is run with `pygame-ce` and `python game.py`.

From your repository folder:

```
pip install pygame-ce
python game.py
```

Then:

1. Start the game.
2. You should see red floor traps in the dungeon.
3. Move using W/A/S/D or arrow keys.
4. Walk directly onto a trap.
5. You should immediately be moved back to the original starting room.
6. The bottom status bar should say:
"You stepped on a trap! Returned to start."
7. Try walking onto multiple traps to verify they all work.

8. Collect the key and verify key collection still works.
9. Reach the chest with the key and confirm the existing win condition works.
10. Restart with R and verify the game still generates a new dungeon and traps.

One intentional detail: stepping on a trap resets the player's position but does not remove a previously collected key, because the Task 1 requirement specifies resetting the player, while the existing key state is otherwise left untouched. This avoids changing existing key/chest functionality.

Task 2 — Enemy Guard

USER / PROMPT

USER / PROMPT Implement Task 2 from the README in the existing game.py.

First inspect the current code and preserve the trap functionality and all existing gameplay.

Add an enemy guard near the chest with the following behaviour:

- The guard should be visibly represented in the dungeon.
- It should patrol continuously back and forth between two defined patrol points.
- The guard should move automatically each frame.
- If the player collides with/touches the guard, reset the player to the original starting position.
- Display a clear status message when the guard collision resets the player.
- Make sure the guard does not break walls or existing tile collision.
- Preserve key collection, chest interaction, traps, restart, and player movement.
- Do not rewrite unrelated parts of the project or add unnecessary dependencies.

After implementing it, explain the changes and give me a short checklist for testing the guard patrol and collision behaviour.

ASSISTANT / RESPONSE I've confirmed the current file already contains the Task 1 trap implementation. For Task 2, I'm adding a small Guard class and deriving its patrol points from the chest room so it stays on valid floor tiles.

The guard is now tied into the normal GameEngine.update() loop, so it advances every frame while the player retains the existing movement and tile collision.

Implemented Task 2 — Enemy Guard in the existing game.py, while preserving the Task 1 trap implementation and the original game architecture.

What I changed

- Added a Guard class rather than rewriting the player/game architecture.
- Added a visible red guard in the dungeon.
- The guard patrols continuously between two points beside the chest.
- The guard moves automatically every game frame.
- Guard movement checks the same grid/wall system as the player, so it cannot move through walls.
- Added player/guard rectangle collision:
 - Player is returned to start.
 - Status becomes:
"The guard caught you! Returned to start."
- Guard is recreated when pressing R, so restart still works.
- Existing functionality remains intact:

- Player movement
- Wall collision
- Key collection
- Chest interaction
- Task 1 traps
- Trap reset message
- Restart

Updated file

[Download the updated game.py](sandbox:/mnt/data/game.py)

I also ran a Python syntax check successfully. I couldn't perform a full graphical runtime test in this environment because pygame isn't installed here, so the actual gameplay test should be done on your machine where you've installed pygame-ce.

Short testing checklist

1. Run:
python game.py
2. Find the red guard near the chest.
3. Leave the game running and watch it move back and forth.
4. Confirm it reverses direction at both patrol points.
5. Walk into the guard.
6. Confirm:
 - Player returns to the starting position.
 - Message says "The guard caught you! Returned to start."
7. Walk into a trap and confirm the existing trap reset still works.
8. Collect the key and confirm it still works.
9. Reach the chest with the key and confirm the existing win condition works.
10. Press R and confirm the dungeon, traps, guard, and starting position reset.

Trap Path Fix

USER / PROMPT

USER / PROMPT[User provided a screenshot of the running game.]

impossible for me to get past the trap, change the settings such that it doesnt ever fully block my way and there is actually a way out

ASSISTANT / RESPONSEYes — the issue in your screenshot is that two traps spawned next to each other in a narrow corridor, effectively creating a wall.

I changed the trap placement so this cannot happen in the important sense:

- The game first finds a valid path from the starting position to the chest.
- Traps are never placed on that path.
- You still get 4 traps elsewhere in the dungeon.
- The existing trap collision/reset behaviour is unchanged.
- Guard, key, chest, movement, and restart are untouched.

I also verified the modified file passes Python syntax validation.

Updated file

[Download the fixed game.py](sandbox:/mnt/data/game.py.fixed)

Replace your current game.py with this file.

Then run:

```
python game.py
```

You should now always have an actual route from the starting area toward the chest, even when traps are generated.

Task 3 — Mini-map

USER / PROMPT

USER / PROMPT Implement Task 3 from the README in the existing game.py.

Do not rewrite the existing game. Preserve all currently working functionality including traps, the enemy guard, player movement, key collection, chest interaction, and restart.

Add a small mini-map in one corner of the game window.

Requirements:

- The mini-map must represent the actual dungeon/grid.
- Clearly distinguish walls from walkable floor.
- Show the player's current position on the mini-map.
- Update the player's position in real time as they move.
- Keep the mini-map small enough that it does not interfere with normal gameplay.
- Use the existing dungeon/grid data rather than creating a separate hard-coded map.
- Keep the implementation understandable and modular.
- Do not add external dependencies.

After implementing it, explain how the mini-map is generated from the existing dungeon data and how I can test that it stays synchronized with the player.

ASSISTANT / RESPONSE Implemented Task 3 — Mini-map without changing the existing game architecture.

What changed

Added a draw_minimap() method to GameEngine.

The mini-map:

- Uses the existing self.grid directly.
- Shows:
 - Dark tiles = walls
 - Light tiles = walkable areas
- Shows the player as a blue dot.
- Uses the player's actual pixel position, converted to mini-map coordinates, so it updates continuously as the player moves.
- Is a compact 160 × 120 px map in the top-right corner.
- Gets regenerated automatically when R creates a new dungeon because it always reads the current self.grid.
- Does not introduce any dependencies.

Updated file

[Download game_task3.py](sandbox:/mnt/data/game_task3.py)

Replace your current game.py with this file and run:

```
python game.py
```

Testing checklist

1. Start the game.
2. Look at the top-right corner for the mini-map.
3. Check that the dungeon shape on the mini-map matches the actual dungeon.
4. Move the player around with WASD/arrow keys.
5. Confirm the blue player marker moves immediately with you.
6. Walk into a wall and confirm the player doesn't move through it while the mini-map remains synchronized.
7. Step on a trap and confirm the player returns to the start and the mini-map marker also returns to the start.
8. Get caught by the guard and confirm the marker returns to the starting position.
9. Press R and confirm the new dungeon produces a corresponding new mini-map.

Task 4 — Inventory HUD

USER / PROMPT

USER / PROMPT Implement Task 4 from the README in the existing game.py.

Preserve all existing functionality and do not rewrite unrelated code.

Add an inventory HUD/slot that:

- Is visible during gameplay.
- Shows an empty inventory slot when the game starts.
- Displays a clear key icon or key representation when the player collects the key.
- Updates immediately when the key is collected.
- Uses the existing key state from the game rather than creating a duplicate key system.
- Remains visible after collection.
- Works correctly when the game is restarted.
- Does not interfere with the mini-map, traps, guard, player movement, or chest interaction.

Keep the implementation simple and modular and do not add unnecessary dependencies.

After implementing it, explain what was changed and provide a final testing checklist covering Tasks 1–4.

ASSISTANT / RESPONSE Implemented Task 4 — Inventory HUD while preserving Tasks 1–3.

What I changed

Added a small inventory slot in the bottom HUD.

- At game start: the slot is empty.
- After collecting the key: a yellow key icon appears immediately.
- The inventory uses the existing:
self.player.has_key
so there is no duplicate key/inventory state.
- The inventory remains visible after the key is collected.
- Pressing R calls the existing reset() method, which creates a new Player with has_key = False, so the inventory correctly becomes empty again.
- The mini-map remains in the top-right and is unaffected.
- Traps, guard, movement, chest interaction, and restart are unchanged.

Updated file

[Download game_task4.py](sandbox:/mnt/data/game_task4.py)

Replace your current game.py with this file.

Run:

```
python game.py
```

Final testing checklist — Tasks 1–4

Task 1 — Traps

- [] Traps are visibly shown on the floor.
- [] Walking onto a trap returns the player to the start.
- [] Message says "You stepped on a trap! Returned to start."
- [] There is always a usable route toward the chest.

Task 2 — Guard

- [] Guard is visible near the chest.
- [] Guard slowly patrols back and forth.
- [] Guard doesn't walk through walls.
- [] Touching the guard returns the player to the start.
- [] Guard collision message appears.

Task 3 — Mini-map

- [] Mini-map is visible in the top-right.
- [] Walls and floors are distinguishable.
- [] Player marker moves as the player moves.
- [] Player marker returns to the start after a trap/guard collision.
- [] Pressing R generates a mini-map matching the new dungeon.

Task 4 — Inventory

- [] Empty inventory slot is visible when starting.
- [] Collecting the key immediately displays the key icon.
- [] Key icon remains visible while playing.
- [] Chest still works after collecting the key.
- [] Pressing R clears the key and returns the inventory slot to empty.

Conversation Outcome

The conversation progressed through four requested game features while preserving the existing game architecture: **floor traps, enemy guard, mini-map, and inventory HUD.**

- Task 1 — Floor traps with reset-to-start behaviour and a guaranteed route toward the chest.
- Task 2 — A visible, slower patrolling guard with collision reset.
- Task 3 — A live mini-map generated from the actual dungeon grid.
- Task 4 — An inventory slot driven by the existing `player.has_key` state.